

Data Wrangling & Exploration in Python

Using the GSS Subset Dataset

Victor Wandera Lumumba, MSc.
Statistician — Data Analyst — ML Expert

Mediacrest Training College

June 22, 2026

Learning Objectives

- Import and inspect real-world tabular data.
- Identify and handle missing values, duplicates, and outliers.
- Subset and filter rows based on logical conditions.
- Select specific columns or index ranges efficiently.
- Group data and compute aggregations.
- Engineer a new categorical income variable.
- Generate comprehensive descriptive statistics.

The Dataset: GSS Subset

- **File:** `GSSsubset.csv`
- **Records:** ~3,000 respondents from the General Social Survey.
- **Columns:** `id`, `sex`, `degree`, `income`, `marital`, `age`, `height`, `weight`, `hrswrk`.

1. Data Importation

- CSV is the universal standard for tabular data.
- `read_csv()` handles parsing, type inference, and header alignment.
- Key parameters: `sep`, `header`, `na_values`, `dtype`.

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the dataset
5 df = pd.read_csv('GSSsubset1.csv')
6 df.head()
```

2. Initial Data Inspection: Info & Describe

- `.info()`: Reveals memory footprint, data types, and missing values.
- `.describe()`: Gives numeric summaries (mean, std, min, max, quartiles).

```
1 # Data types, non-null counts, memory usage
2 df.info()
3
4 # Quick statistical summary for numeric columns
5 df.drop(columns=["id"]).describe()
```

2. Initial Data Inspection: Frequencies

- `.value_counts()`: Shows category frequencies.
- Use `normalize=True` to get percentages.

```
1 # Check categorical value distributions
2 print(df['degree'].value_counts())
3 print(df['marital'].value_counts())
4
5 # Get percentages
6 df['sex'].value_counts(normalize=True) * 100
```

Data Cleaning

3. Missing Values: Detection & Impact

- Missing data appears as NaN, None, or placeholders.
- **Impact:** Skews statistics, breaks algorithms, reduces sample size.
- **Strategy:** Detect → Decide (Drop / Impute / Flag).

```
1 # Count missing per column
2 df.isna().sum()
3
4 # Percentage of missing data
5 df.isna().mean().round(4) * 100
```


3. Missing Values: Visualizing Missingness

- Visual checks help identify patterns in missing data.
- **Heatmaps:** `sns.heatmap` or `missingno` show where data is missing.
- **Bar Charts:** Highlight variables with the highest missingness.
- Helps decide whether to drop or impute based on patterns.

Diagnostic Tip

If missingness is random (MCAR), imputation is safer. If systematic, investigate the cause!

3. Missing Values: Standard Imputation

- **Numeric:** Use Median (safer for skewed data like income/weight).
- **Categorical:** Use Mode.

```
1 # Automatic numeric imputation
2 numeric_cols = df.select_dtypes(include='number').columns
3 df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())
4
5 # Categorical imputation
6 cat_cols = df.select_dtypes(include=['object']).columns
7 for col in cat_cols:
8     df[col] = df[col].fillna(df[col].mode()[0])
```

3. Missing Values: ML Approach

- Scikit-Learn's SimpleImputer for robust pipelines.
- Integrates well with machine learning workflows.

```
1 from sklearn.impute import SimpleImputer
2
3 num_cols = df.select_dtypes(include='number').columns
4 imputer = SimpleImputer(strategy='median') # or 'mean'
5
6 # Fit and transform
7 df[num_cols] = imputer.fit_transform(df[num_cols])
```

4. Duplicates: Detection & Removal

- **Exact duplicates:** Identical rows across all columns.
- **Impact:** Inflates counts, biases averages, breaks unique constraints.

```
1 # Count exact duplicate rows
2 dup_count = df.duplicated().sum()
3 print(f"Exact duplicate rows found: {dup_count}")
4
5 # Remove duplicates (keeps first occurrence)
6 df_clean = df.drop_duplicates()
```

4. Duplicates: Uniqueness & Primary Keys

- Check uniqueness of primary keys (e.g., id).
- Near-duplicates: same ID with minor typos.

```
1 # Check uniqueness of primary key (id)
2 unique_ids = df['id'].nunique()
3 total_records = len(df)
4
5 print(f"ID uniqueness: {unique_ids} unique / {total_records} total")
6
7 # If IDs should be unique but aren't, investigate survey re-submissions
```

5. Outliers: Detection using IQR

- **Outliers:** Values far outside the expected range.
- **Detection:** IQR method ($Q1 - 1.5 \times IQR$, $Q3 + 1.5 \times IQR$).

```
1 Q1 = df[col].quantile(0.25)
2 Q3 = df[col].quantile(0.75)
3 IQR = Q3 - Q1
4
5 lower_bound = Q1 - 1.5 * IQR
6 upper_bound = Q3 + 1.5 * IQR
7
8 # Identify outliers
9 outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
```

5. Outliers: Handling via Winsorizing

- **Winsorizing:** Caps outliers to preserve sample size.
- **Alternative:** Log transformation for skewed data.

```
1 # Cap outliers instead of removing rows
2 df[col] = np.where(df[col] < lower_bound, lower_bound, df[col])
3 df[col] = np.where(df[col] > upper_bound, upper_bound, df[col])
4
5 # Verify outliers after capping
6 print(f"Column '{col}': {len(outliers)} outliers remaining")
```

Data Manipulation

6. Filtering Rows: Boolean Masking

- Subset data based on logical conditions.
- Combine conditions with & (AND), | (OR), ~ (NOT).

```
1 # Single condition
2 males = df[df['sex'] == 'MALE']
3
4 # Multiple AND conditions
5 subset = df[(df['sex'] == 'FEMALE') &
6             (df['age'] > 35) &
7             (df['marital'] == 'MARRIED')]
```

6. Filtering Rows: .query() & .isin()

- .query(): Uses string expressions, highly readable.
- .isin(): Replaces long | chains for membership testing.

```
1 # Cleaner syntax with query()
2 df_filtered = df.query('age >= 25 and income > 50000')
3
4 # Filter using a list of values
5 degrees = ['BACHELOR', 'GRADUATE', 'JUNIOR COLLEGE']
6 df_edu = df[df['degree'].isin(degrees)]
7
8 # NOT IN with ~ operator
9 df_no_hs = df[~df['degree'].isin(['LT HIGH SCHOOL', 'HIGH SCHOOL'])]
```

7. Selecting Columns: .loc & .iloc

- Extract specific variables for focused analysis.
- .loc: Label-based selection.
- .iloc: Position-based selection.

```
1 # Label-based selection
2 df_labels = df.loc[:, 'age':'weight']
3
4 # Position-based selection
5 df_positions = df.iloc[:, [0, 2, 3, 7]] # id, degree, income, weight
6
7 # Combine selection + filtering in one efficient operation
8 df_final = df.loc[df['age'] > 25, ['sex', 'degree', 'income']]
```

7. Selecting Columns: Dropping & Type Selection

- Drop unwanted columns to save memory.
- Use `select_dtypes` to group numeric vs categorical.

```
1 # Drop unwanted columns
2 df_dropped = df.drop(columns=['id', 'height'])
3
4 # Filter columns by data type
5 numeric_cols = df.select_dtypes(include='number').columns.tolist()
6 categorical_cols = df.select_dtypes(include='object').columns.tolist()
7
8 print(f"Numeric: {numeric_cols}")
9 print(f"Categorical: {categorical_cols}")
```

8. Grouping By: Split-Apply-Combine

- Split data by category, apply function, combine results.
- Essential for comparative analysis (e.g., income by degree).

```
1 # Mean income by education level
2 edu_income = df.groupby('degree')['income'].mean()
3
4 # Group by multiple columns
5 grouped = df.groupby(['sex', 'marital'])['income'].mean()
6
7 # Unstack for a pivot-table-like view
8 grouped.unstack()
```

8. Grouping By: Multiple Aggregations

- `.agg()` lets you run multiple statistics simultaneously.
- Flattening column names prevents `KeyError` issues downstream.

```
1 summary = df.groupby('degree').agg({
2     'income': ['mean', 'median', 'std', 'count'],
3     'age': 'mean',
4     'hrswrk': 'sum'
5 })
6
7 # Flatten multi-level column names
8 summary.columns = ['_'.join(col) for col in summary.columns]
```

Feature Engineering & Statistics

9. Feature Engineering: np.where

- Create new variables from existing ones.
- Convert continuous income to categorical `Income_Level`.

```
1 median_income = df['income'].median()
2
3 # np.where (fast, vectorized) - Recommended for binary splits
4 df['Income_Level'] = np.where(
5     df['income'] >= median_income,
6     'High Income',
7     'Low Income'
8 )
9
10 print(df['Income_Level'].value_counts())
```


9. Feature Engineering: pd.cut

- `pd.cut()` is ideal for ordered categorical bins with explicit edges.
- Automatically handles NaNs and preserves order for sorting.

```
1 # Define bins and labels
2 bins = [0, median_income, np.inf]
3 labels = ['Low Income', 'High Income']
4
5 # pd.cut (handles edges cleanly, preserves order)
6 df['Income_Level_cut'] = pd.cut(
7     df['income'], bins=bins, labels=labels, include_lowest=True
8 )
9
10 # Verify order
11 print(df['Income_Level_cut'].cat.ordered)
```

10. Descriptive Statistics: Overall Summaries

- **Numeric:** mean, median, std, skewness, kurtosis.
- **Categorical:** frequencies, mode, unique counts.

```
1 # Numeric summary
2 df.describe()
3
4 # Categorical summary
5 df.describe(include=['object', 'category'])
6
7 # Distribution shape metrics
8 print(f"Income Skewness: {df['income'].skew():.3f}")
9 print(f"Income Kurtosis: {df['income'].kurtosis():.3f}")
```

10. Descriptive Statistics: Grouped Summaries

- Grouped summaries reveal hidden patterns.
- Coefficient of Variation (CV) for relative variability.

```
1 # Summary by Income Level
2 grouped_stats = df.groupby('Income_Level').agg({
3     'age': ['mean', 'std'],
4     'hrswrk': 'mean'
5 })
6
7 # Coefficient of Variation (CV = std/mean)
8 for col in ['income', 'age', 'weight']:
9     cv = df[col].std() / df[col].mean()
10    print(f"{col} CV: {cv:.3f} ({cv*100:.1f}%)"
```

11. Correlation Analysis: Matrix

- Examine linear relationships between continuous variables.

```
1 # Select numeric columns for correlation
2 numeric_vars = ['age', 'height', 'weight', 'hrswrk', 'income']
3 corr_matrix = df[numeric_vars].corr()
4
5 print(corr_matrix.round(3))
```

Note

The correlation matrix is symmetric, with 1.0 on the diagonal.

11. Correlation Analysis: Interpretation

- Highlight strong correlations (e.g., $|r| > 0.3$).
- **Example:** Height $\leftrightarrow$ Weight often shows a strong positive correlation.
- **Caveat:** Correlation $\neq$ Causation!
- Always consider context and confounding variables.

Important

A high correlation does not imply that one variable causes the other to change.

12. Exporting Cleaned Data

- Always save your cleaned/processed data for reproducibility.
- Document your decisions for auditability.

```
1 # Create a final cleaned dataset
2 df_export = df.copy()
3
4 # Add derived variables (e.g., BMI)
5 df_export['BMI'] = (df_export['weight'] * 0.453592) / \
6                     ((df_export['height'] * 0.0254) ** 2)
7
8 # Save to CSV
9 df_export.to_csv('GSSsubset_processed.csv', index=False)
```

Summary & Key Takeaways

- Proper import and inspection prevent downstream errors.
- Handle missing values, duplicates, and outliers systematically.
- Use boolean masking and `.query()` for efficient filtering.
- Grouping and feature engineering unlock deeper insights.
- Always export and document your cleaned data!

Questions?

Victor Wandera Lumumba

Statistician — Data Analyst — ML Expert

Email: lumumbavictor172@gmail.com

Web: beyonddataanalytics.online

GitHub: [Lumumba1992](https://github.com/Lumumba1992)